Práctica 3. El sensor de Humedad.

Objetivos

El objetivo de esta práctica es comunicar el convertidor ADS1115 con el microcontrolador ESP8266 utilizando el bus de comunicaciones I2C para realizar un sensor de humedad.

- Montar en una placa de prototipos un microcontrolador al que le conectaremos la placa con el convertidor ADS1115 y lo utilizaremos para conectar un sensor de humedad
- Escribir un pequeño programa que nos permitirá detectar la humedad y presentar su valor a través del monitor serie
- Testar el funcionamiento del sistema implementado y hacer las modificaciones oportunas para presentar la lectura indicando el porcentaje de humedad que capta el sensor. Siendo 100% cuando el sensor está sumergido en agua y 0% cuando está totalmente seco.

Material

Todo el material se proporciona en la sesión de prácticas. Cada grupo de prácticas tendrá:

- Un microcontrolador ESP8266
- Un sensor de humedad SEN0193
- Un convertidor analógico digital ADS1115
- Cables Dupont macho-macho

Introducción

Antes de comenzar montar el circuito debemos conocer el patillaje del Microcontrolador ESP8266. Para ello se proporciona, en el anexo 1, la hoja de características de este microcontrolador. Debéis tener siempre presente esta hoja ya que es la que os indica a qué corresponde cada uno de los pines del microcontrolador.

Este microcontrolador sólo posee un convertidor analógico digital por lo que, si queremos usar más de un sensor analógico, tenemos que utilizar un convertidor externo. En nuestro caso

vamos a hacer uso del convertidor ADS1115, cuya hoja de características se proporciona en el anexo 2.

El convertidor ADS1115

La Figura 1 muestra el patillaje del este convertidor.

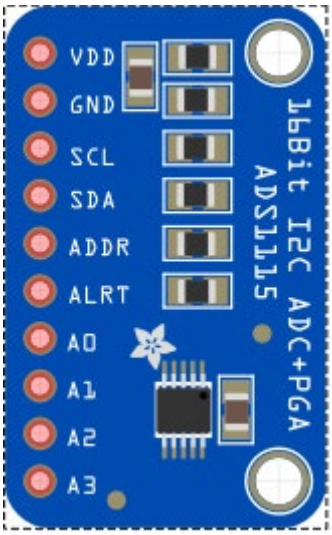


Figura 1. ADS1115

Características del ADS1115

- La alimentación (VDD) se realiza a 5V
- La comunicación con el microcontrolador se hace a través del bus I2C. Repasar los apuntes de clase en los que se explica cómo funciona este bus. Se utilizarán los pines SCL para el reloj y SDA para los datos.
- El pin ADDR se utiliza para proporcionar la dirección de comunicación. Si se conecta a GND la dirección será la 0x48, si se conecta a 5V la dirección será 0x49.
- Posee cuatro canales de entrada (A0, A1, A2 y A3) de manera que vamos a poder utilizarlo para captar señales de cuatro sensores analógicos
- Codifica la señal digital con 16 bits, siendo el rango de salida para señales unipolares [0, 2^{15}-1]; y para señales bipolares [-(2^{15}-1), 2^{15}-1[;
- Permite realizar un ajuste de la ganancia, de forma que el rango de entrada se puede ajustar a los valores indicados en la tabla 1.

Atributo	Rango de la señal analógica	Resolución
GAIN_ONE	±4.096 V	2mV
GAIN_TWO	±2.048 V	1mV
GAIN_FOUR	±1.024 V	0.5mV
GAIN_EIGHT	±0.512 V	0.25mV

Conexión con el Microcontrolador ESP8266

Para conectar el microcontrolador al ADS1115 a través del bus I2C deberíamos realizar las conexiones indicadas en la Figura 2. En esta figura se han conectado dos convertidores, por lo que con este montaje podríamos leer hasta 8 sensores.

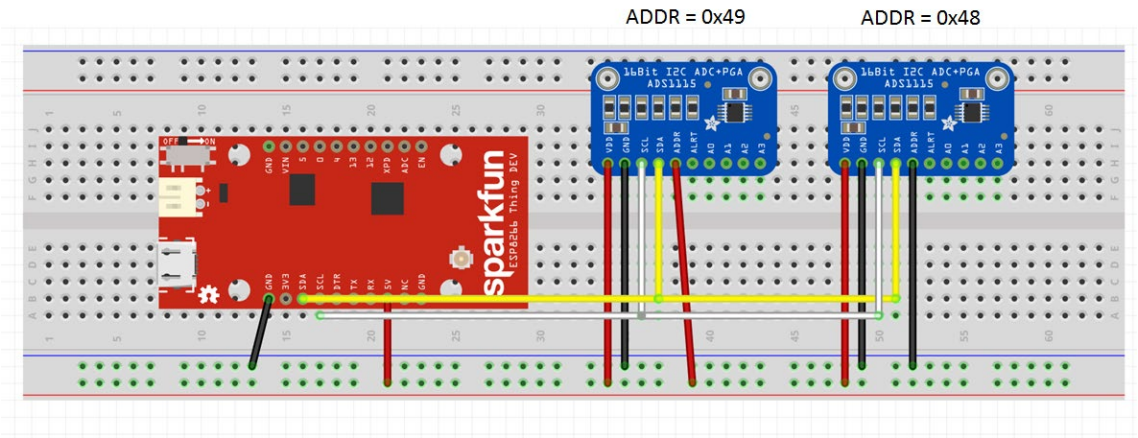


Figura 2. Conexión del microcontrolador ESP8266 a dos ADS1115 usando el bus I2C

Para facilitar las conexiones se ha implementado una placa de interconexiones que permitirá conectar el ADS1115 apilado sobre el ESP8266. En la Figura 3 se muestra el esquema de la placa de interconexiones, una foto de la placa con el convertidor incluido y diferentes vistas de la conexión del microcontrolador con el ADS1115 a través de la placa.

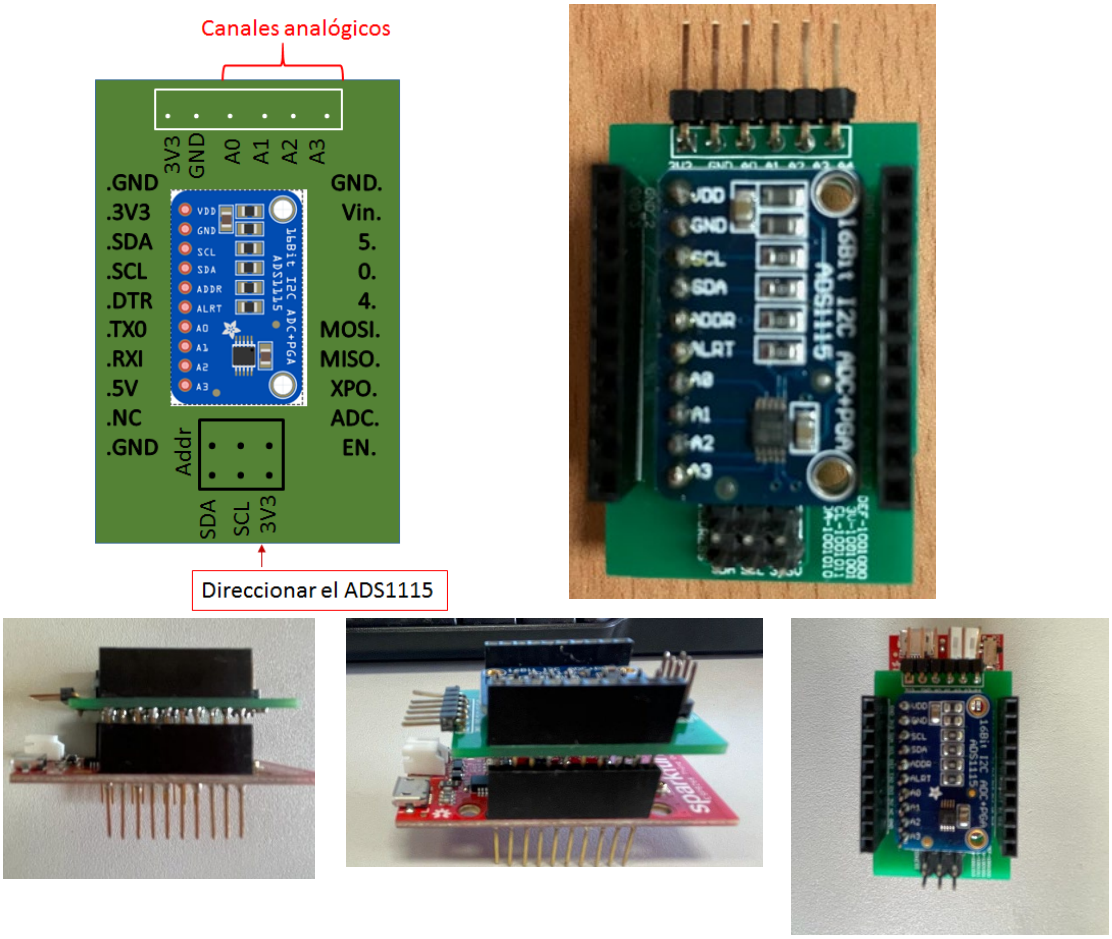
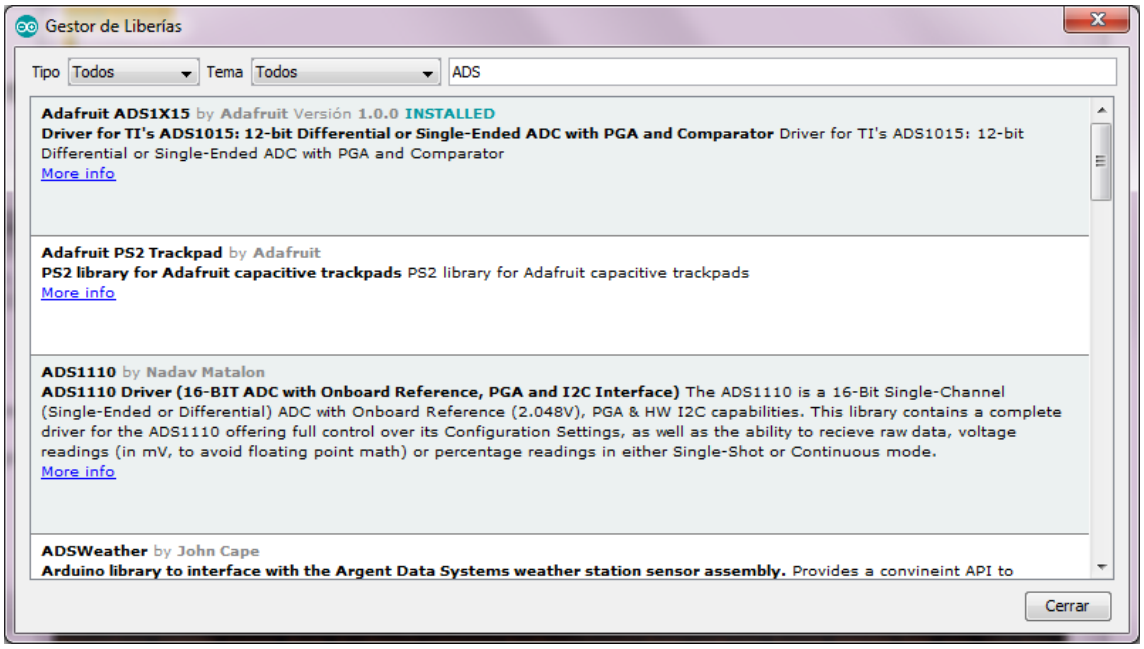


Figura 3. Placa de interconexiones, foto de la placa con el convertidor ADS1115 y tres vistas de la conexión ESP8266 con el ADS1115 a través de la placa de interconexiones

Para poder utilizar este convertidor es necesario instalarse la **librería Adafruit ADS1X15** usando el Gestor del Librerías incluido en el IDE de Arduino. **Menú Programa> Incluir Librerías > Gestor de Librerías.**



Las **funciones** que utilizaremos para trabajar con este convertidor las podéis encontrar en la página el anexo 2, apartado pro, en el apartado Programming. Consultar este anexo y obtener qué funciones se utiliza para:

- Construir el objeto del convertidor
- Inicializar el convertidor
- Ajustar la ganancia del convertidor
- Leer de uno de los canales una señal analógica unipolar

El sensor de humedad SEN0193

Es un sensor capacitivo que mide la humedad en el entorno. Se ha diseñado para medir la humedad en el suelo, pero podemos utilizarlo para medir la humedad en la tubería del huerto hidropónico.

Las características del sensor son las siguientes:

- Alimentación entre 3.3V y 5V (Cable rojo de la Figura 4)
- Tensión de salida máxima 3V (Cable azul de la Figura 4)
- Corriente de operación 5mA



Figura 4. Sensor de humedad SEN0193

Al tratarse de un sensor analógico lo conectaremos a uno de los canales del ADC ADS1115.

Apartado 1. Implementación del Circuito

En primer lugar, montaremos el circuito de la Figura 5 en la placa de prototipos. Recordad que, en vuestro caso, el ADS1115 está conectado a través de la placa de interconexiones, así que tendréis que conectarlo encima del microcontrolador ESP8266 tal y como se muestra en la Figura 3.

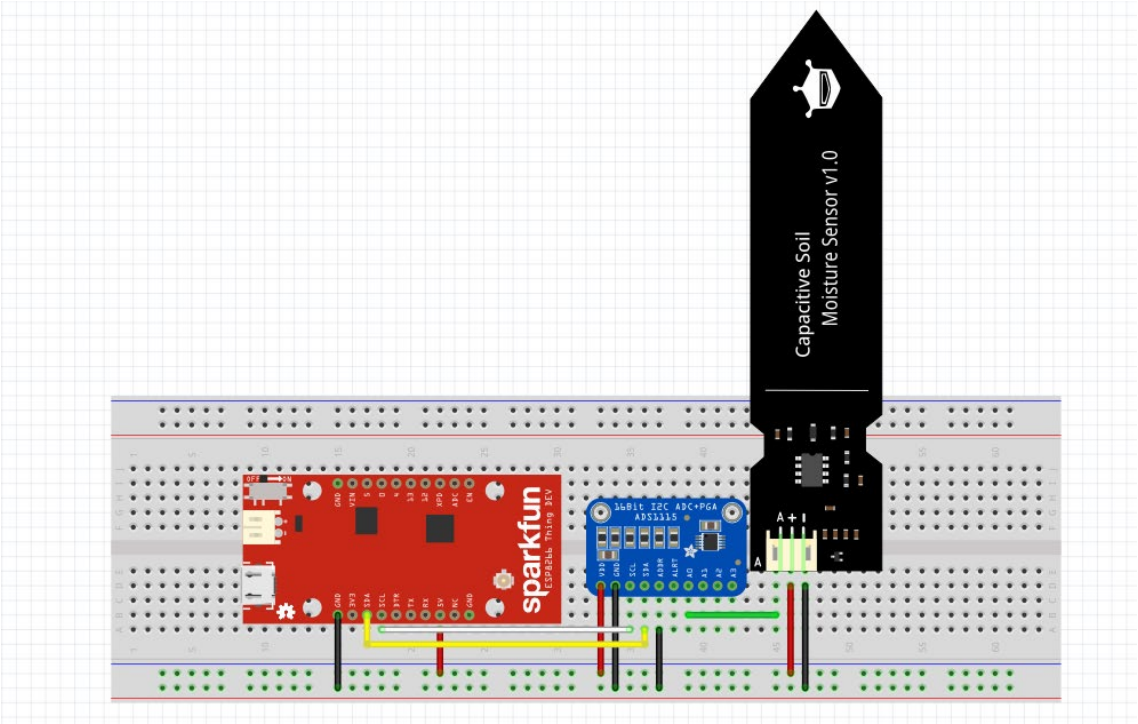


Figura 5. Circuito en la placa de prototipos

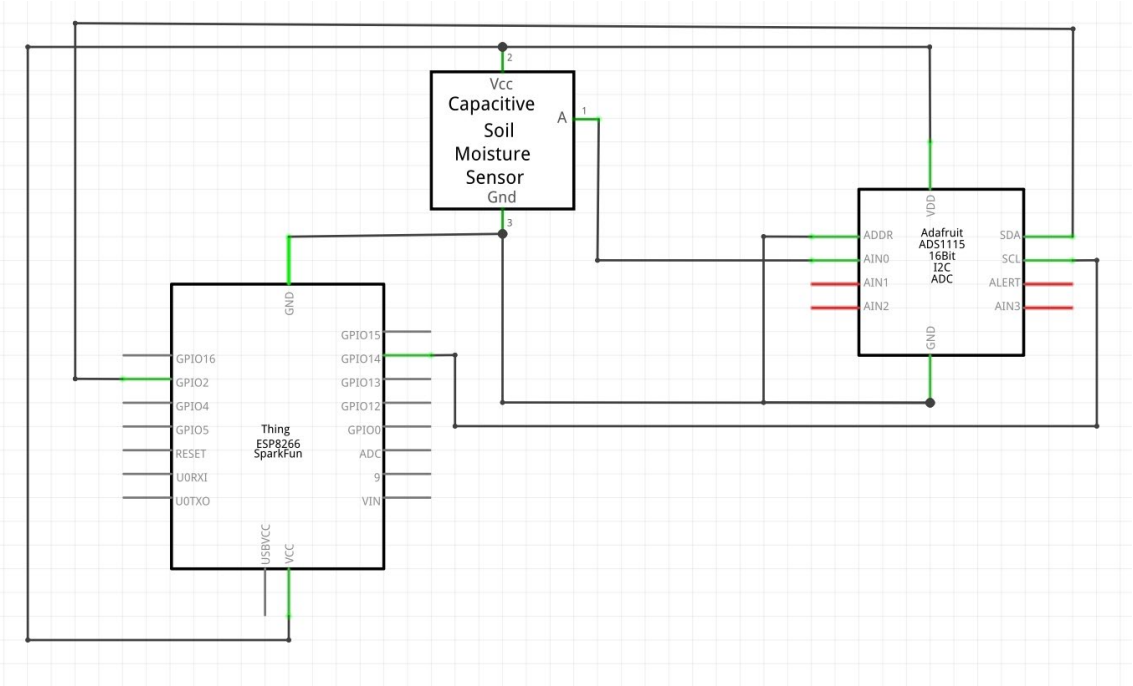


Figura 6 Esquemático del circuito de la Figura 1

Apartado 2. Escribir el código en el IDE de Arduino

Abrimos el IDE de Arduino y completamos el siguiente programa para poder leer el valor de humedad del sensor. Tendréis que incluir:

- La definición del constructor del objeto ADS1115
- La inicialización del ADS1115
- El ajuste de la ganancia (pensad qué valor de ganancia sería más adecuada para la entrada analógica que vais a tener)
- La función de lectura del canal del ads1115

Todas estas instrucciones las tenéis tanto en la explicación del ADS1115 de PoliformaT como en la hoja de características que se incluye en el anexo 2 de esta práctica.

En el programa se han introducido comentarios para indicar en qué lugar hay que introducir cada uno de los elementos anteriores.

```
/************************************************************
 * Practica 3: ADs1115 + Sensor de Humedad
 ************************************************************/

/* Incluir la librería Adafruit_ADS1015.h*/


int channelValue = 0;     // canal del ADS1115 por el que mediremos la humedad
int sensorValue = 0;  // variable para almacenar el valor digital leido
int humidityValue = 0;  // variable para almacenar el valor de humedad

/* Definir el constructor del objeto ads1115, con dirección 0x48*/

/* Configuracion del sistema*/
void setup() {
  Serial.begin(9600); // inicializamos la com. serie (UART) a 9600 baudios
  /*  inicializamos el ADS1115 */

  /* Ajustamos la ganancia */

}


void loop() {
  // leemos el valor del ADC
  /*Leemos del canal channelValue a través del ADS1115*/
  sensorValue =

  /* Mapeamos el valor leído para obtener % de humedad */
  humidityValue =

  // Imprimimos el valor leido
  Serial.println();
  Serial.print("Humedad: ");
  Serial.print(humidityValue,DEC); // DEC para que muestre la salida en decimal
  // retardo 1 seg
  delay(1000);
}
```

Figura 7 IDE de Arduino con el programa de lectura del ADS1115

Al terminar de rellenar el programa debéis compilarlo, corregir los errores, subirlo a la placa y probarlo.

Apartado 3: Calibrado del sensor

El programa de la Figura 7 permite leer valores del sensor de humedad pero no ofrece la lectura perfectamente calibrada. Para calibrar el sistema deberemos tomar unos valores de referencia, e introducir una fórmula en el programa que incluya el calibrado. Vamos a hacerlo a través del siguiente procedimiento:

1- Realizamos una medida con el sensor seco y apuntamos el valor de la lectura digital (sensorValue). Para ello tendréis que modificar el programa anterior para que os ofrezca por el monitor serie este valor. Llamaremos a este valor medidaSeco.
2- Realizamos una medida con el sensor sumergido en agua. ¡¡Cuidado!! No se debe mojar el circuito. Apuntamos el valor de la lectura digital (sensorValue). Llamaremos a este valor medidaMojado.

3- Obtenemos la expresión correspondiente a la función mostrada en la Figura 8. Si no os acordáis de cómo obtener esta expresión podéis consultar este video.

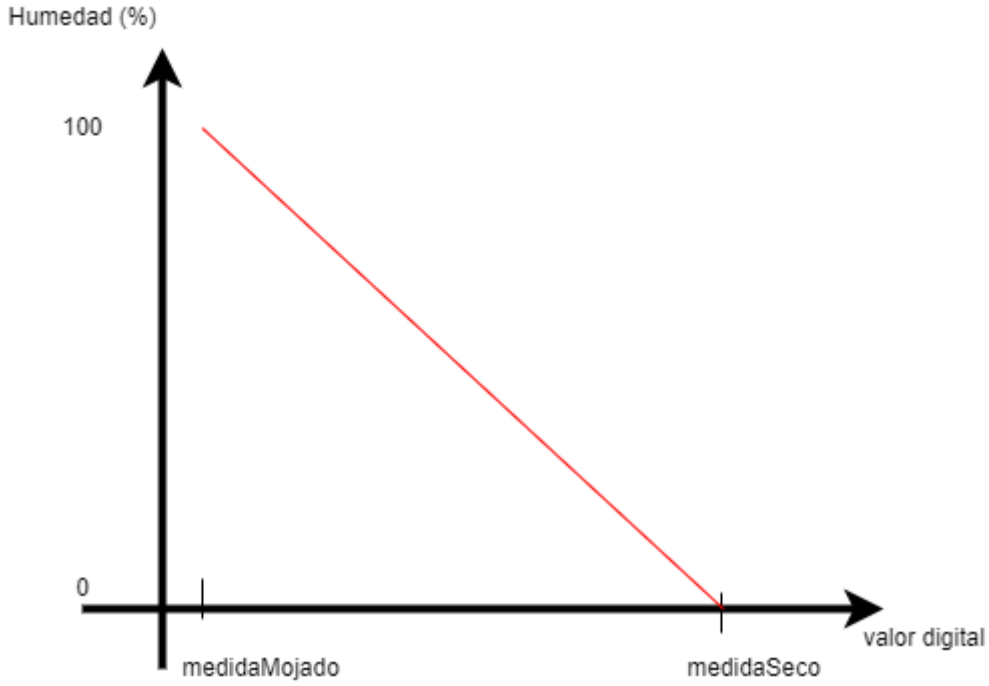


Figura 8. Representación gráfica de la función utilizada para calibrar el sensor de humedad

Incluir aquí los cálculos de la expresión analítica:

4- Una vez obtenida la expresión analítica debéis incluirla en el programa, compilar y probarlo de nuevo.

Apartado 3. Testeo del sistema

Para testear el sistema deberemos realizar varias medidas, con diferentes niveles de agua, e ir tomando nota para rellenar la gráfica de la Figura 12.

Nivel de agua (cm)	Valor digital leído	Humedad (%)
1		
2		
3		
4		
5		
6		

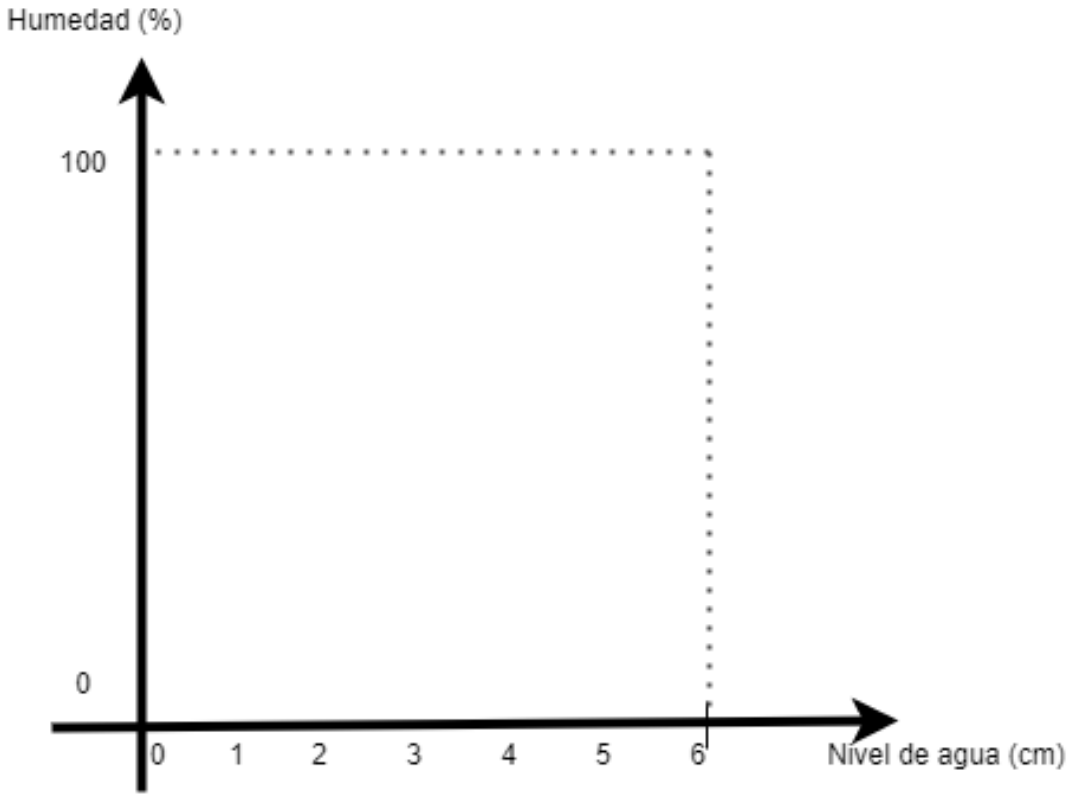


Figura 9. Testeo del sensor de humedad

Apartado 4. Modificando el sistema

Con el siguiente apartado queremos que juguéis un poco con el programa anterior y propongáis cambios al programa. Se premiará a los equipos que consigan las implementaciones más originales.

Algunas ideas:

- Os habréis dado cuenta de que los valores que se leen del sensor de humedad poseen mucha variabilidad. Para hacerlos más precisos podemos tomar varias muestras (por ejemplo 30) y hacer la media de las mismas.
- El sensor de humedad tiende a des-calibrarse con el tiempo. Se podría realizar un programa para guiar al usuario en el calibrado del sensor.

Hoja de Resultados Práctica 3

Al finalizar esta práctica se deberá entregar esta hoja de resultados a la profesora.

Apellidos y Nombre de todos los alumnos/as que han realizado esta práctica:

Fecha:

Apartado 1.

Una vez tengas el circuito implementado, avisa a la profesora para que lo revise y chequee este apartado.

Implementación del circuito. A rellenar por la profesora

1	2	3	4	5

Comentarios:

Apartado 2

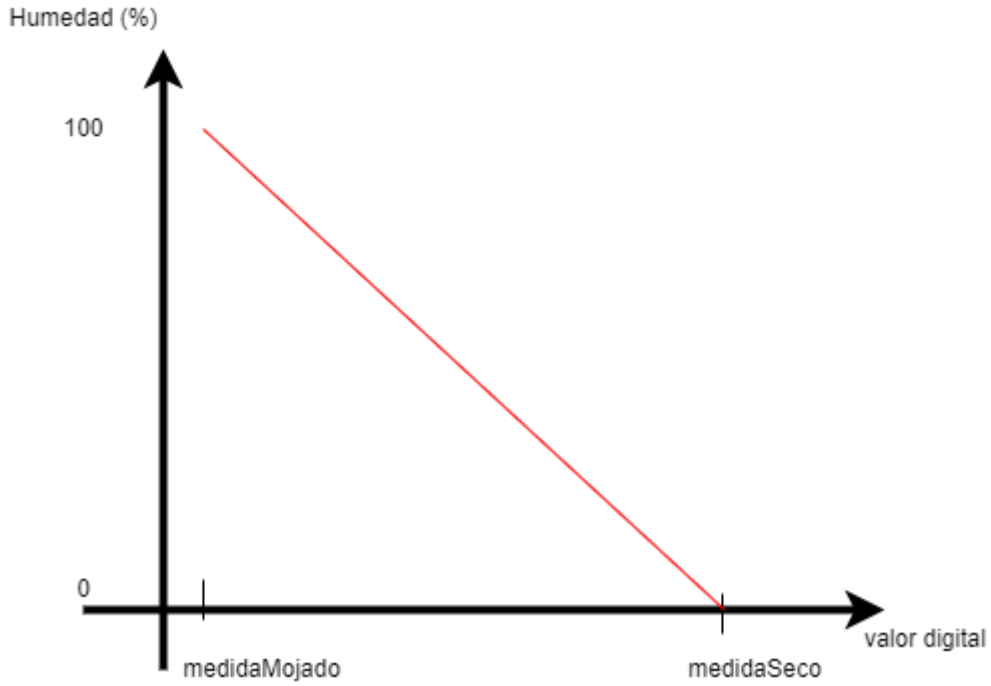
Al terminar el programa avisa a la profesora para que lo revise y chequee este apartado

Constructor del objeto ADS1115	
Inicialización del ADS1115	
Ajuste de la ganancia	
Función de lectura del canal del ads1115	
Mejoras	

Apartado 3

Valor de la medida en Seco =

Valor de la medida en Mojado =

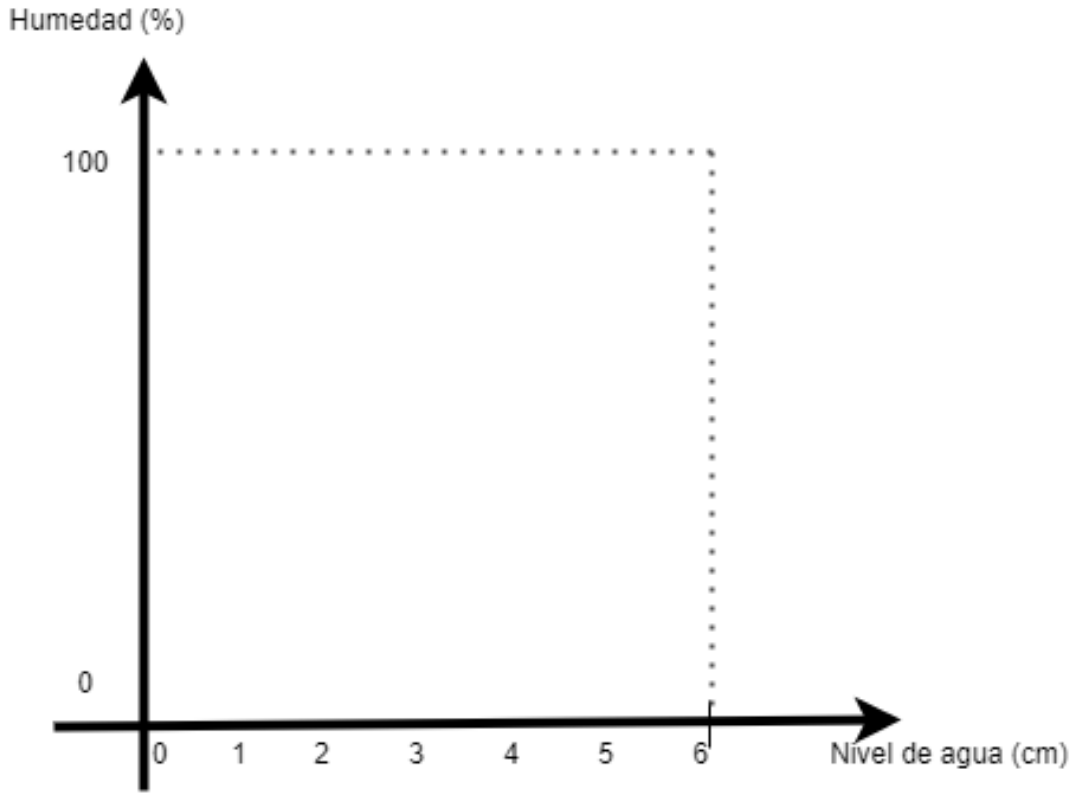


Incluid aquí los cálculos de la expresión analítica:

Realizad el testeo del circuito implementado rellenando esta tabla

Nivel de agua (cm)	Valor digital leído	Humedad (%)
1		
2		
3		
4		
5		
6		

Dibujad la curva obtenida tras el testeo



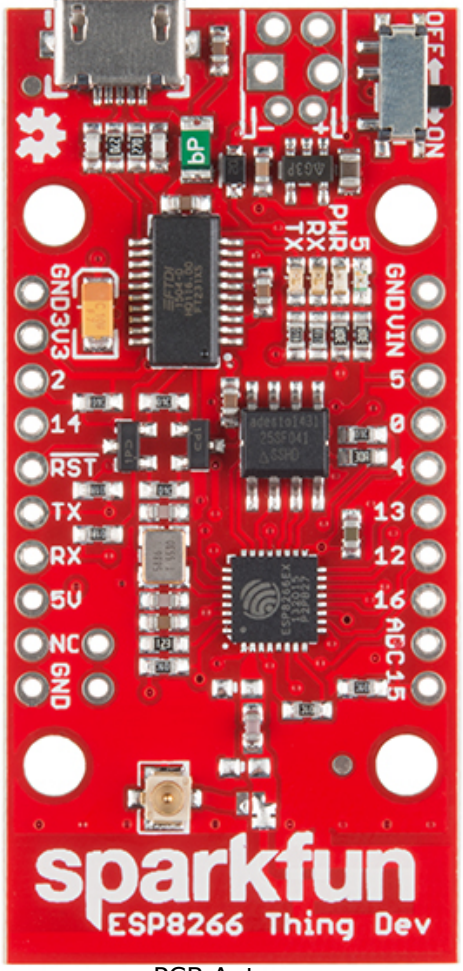
Comentarios: incluid aquí los problemas encontrados

Anexo 1. ESP8266 Thing Dev Board (WRL-13711)

80MHz

Arduino add-on available

Name	Arduino
Power	ADC
GND	Serial
Control	Misc



PCB Antenna

GPIO0
On bootup will run program if high and bootloader if low
Tied to DTR to run bootloader when reset

	GND	GND	GND	GND	
	3.3V	3V3	Vin	Vin	
SDA	D2	2	5	D5	LED
SCL/SCLK	D14	14	0	D0	Used in reset
	Reset	RST	4	D4	
TX	D7	TX	13	D13	MOSI
RX	D8	RX	12	D12	MISO
	V_USB	5V	16	D16	XPD
	Not Connected	NC	ADC	A0	10-bit 1V
	GND	GND	15	D15	CS

u.fl antenna connector
Not Connected
To use swap the solder jumper

Jumpers on back
PWR-LED - cut to disconnect power LED
SLEEP-EN - Connect RST to XPD to enable sleep (disables programming)

Power (ESP8266 Thing)
Vin: 3.7V-6V
VUSB: 5V
VCC:3.3V @500mA
Max 12mA per I/O pin

Typical Power (ESP8266 module)
Transmit 135-215mA
Receive 60-62mA
Standby 0.9mA
Deep sleep 10uA

LEDs
Power: Red
User (pin 5): Blue
RX: Yellow
TX: Green

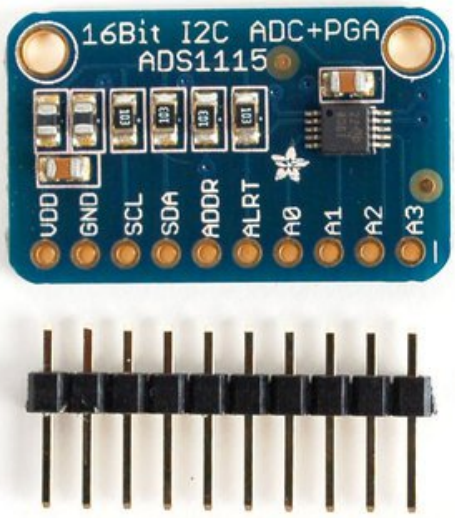
Wi-Fi
802.11 b/g/n
Wi-Fi Direct (P2P) soft AP

sparkfun.com

CC BY SA

Anexo 2. Adafruit 4-Channel ADC Breakouts

Created by Bill Earl



Last updated on 2016-10-04 12:42:36 AM UTC

Guide Contents

Guide Contents 2
Overview 3
ADS1115 Features: 3
ADS1015 Features: 4
Assembly and Wiring 5
Assembly: 5
- Prepare the header strip 5
- Position the breakout board 5
- Solder! 6

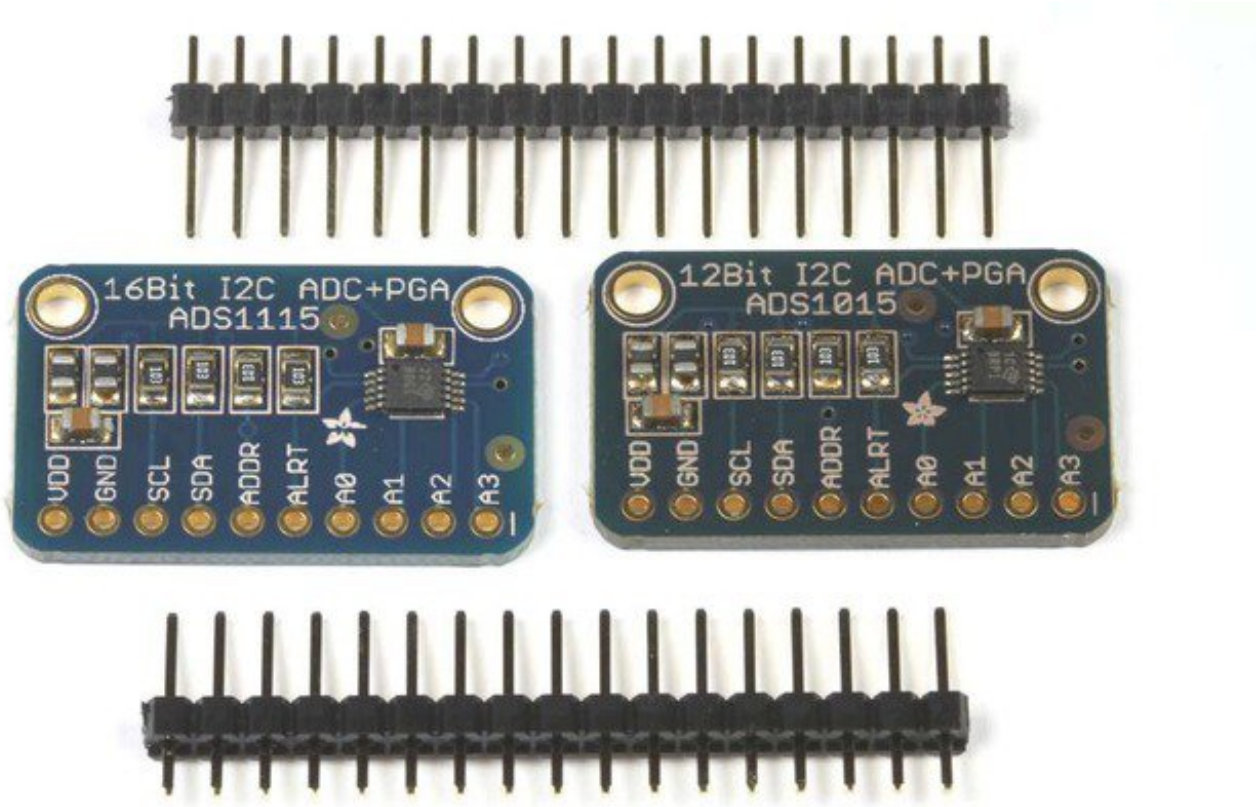
Wiring: 6
- Power 6
- I2C Connections 6
- I2C "Classic" 7
- I2C Addressing 7
- Multiple Boards 8

Signal Connections 10
- Single Ended vs. Differential Inputs: 10
- Which should I use? 10
- Single Ended Connections: 11
- Differential Connections: 11

Programming 13
- Construction and Initialization: 13
- Single Ended Conversion: 13
- Differential Conversion: 14
- Comparator Operation: 15
- Adjusting Gain 16
- Example 17

Downloads 18
Software 18
Files 18
Schematic (Identical For Both) 18
Fabrication Print (Identical For Both) 19

Overview



The ADS1115 and ADS1015 4-channel breakout boards are perfect for adding high-resolution analog to digital conversion to any microprocessor-based project. These boards can run with power and logic signals between 2v to 5v, so they are compatible with all common 3.3v and 5v processors. As many of 4 of these boards can be controlled from the same 2-wire I2C bus, giving you up to 16 single-ended or 8 differential channels. A programmable gain amplifier provides up to x16 gain for small signals.

These two boards are very similar, differing only in resolution and speed. The ADS1115 has higher resolution and the ADS1015 has a higher sample rate.

ADS1115 Features:

- Resolution: 16 Bits
- Programmable Sample Rate: 8 to 860 Samples/Second

- Power Supply/Logic Levels: 2.0V to 5.5V
- Low Current Consumption: Continuous Mode: Only 150µA Single-Shot Mode: Auto Shut-Down
- Internal Low-Drift Voltage Reference
- Internal Oscillator
- Internal PGA: up to x16
- I2C Interface: 4-Pin-Selectable Addresses
- Four Single-Ended or 2 Differential Inputs
- Programmable Comparator

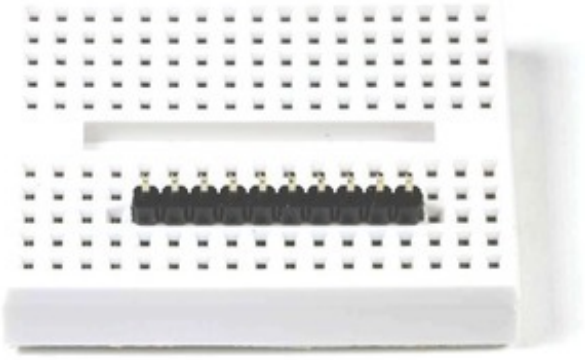
ADS1015 Features:

- Resolution: 12 Bits
- Programmable Sample Rate: 128 to 3300 Samples/Second
- Power Supply/Logic Levels: 2.0V to 5.5V
- Low Current Consumption: Continuous Mode: Only 150µA Single-Shot Mode: Auto Shut-Down
- Internal Low-Drift Voltage Reference
- Internal Oscillator
- Internal PGA: up to x16
- I2C Interface: 4-Pin-Selectable Addresses
- Four Single-Ended or 2 Differential Inputs
- Programmable Comparator

Assembly and Wiring

Assembly:

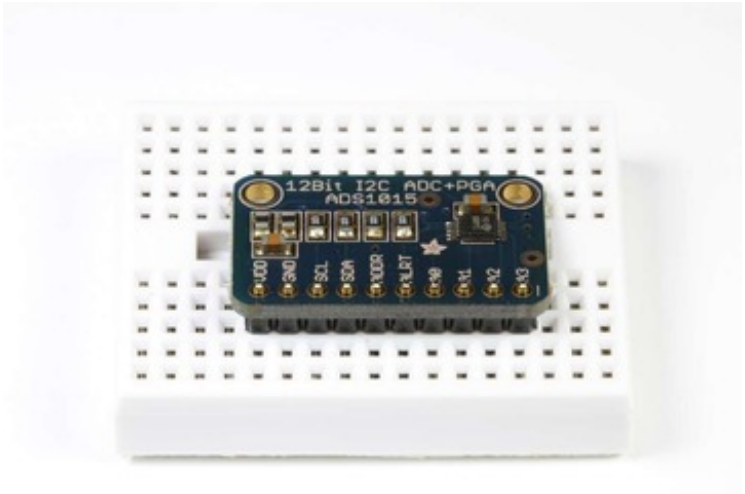
The board comes with all surface-mount parts pre-soldered. For breadboard use, the included header-strip should be soldered on:



Prepare the header strip

Cut the supplied header strip to length and insert it long-pins-down in your breadboard to hold it for soldering.

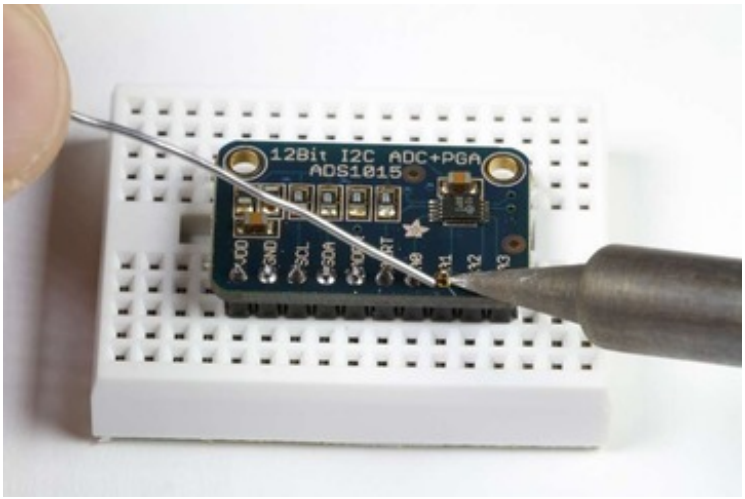
-



Position the breakout board

Place the breakout board on the header pins.

-



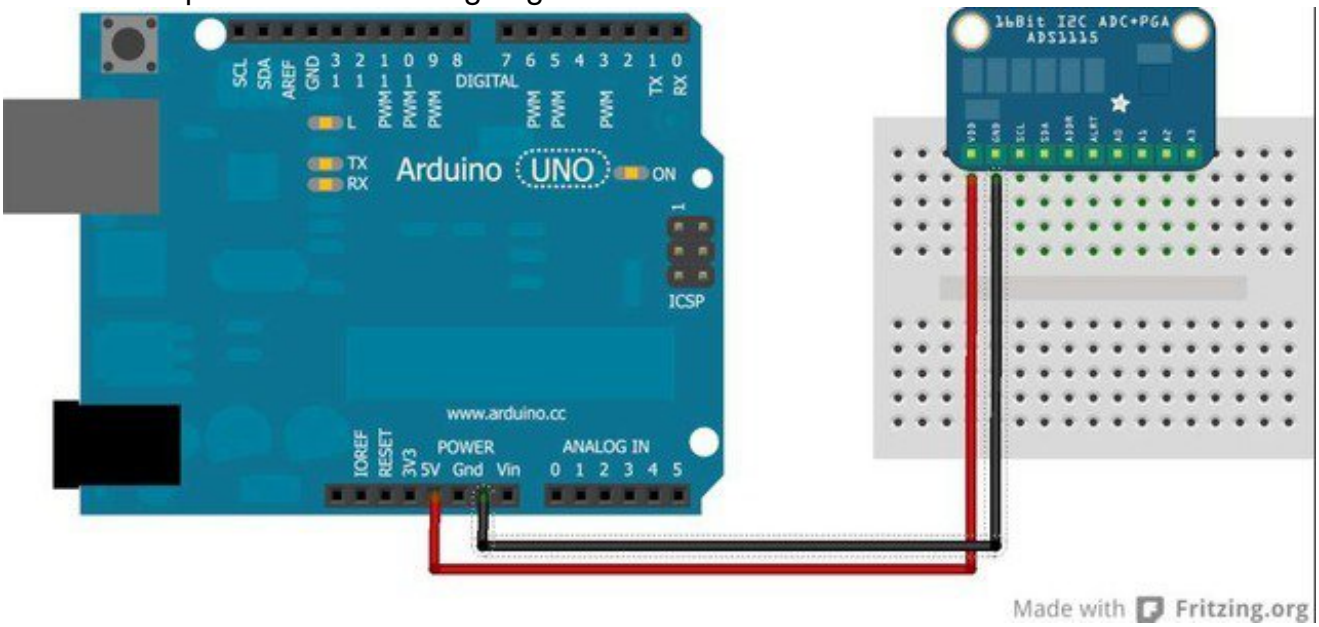
Solder!

Solder each pin for a good electrical connection.

Wiring:

Power

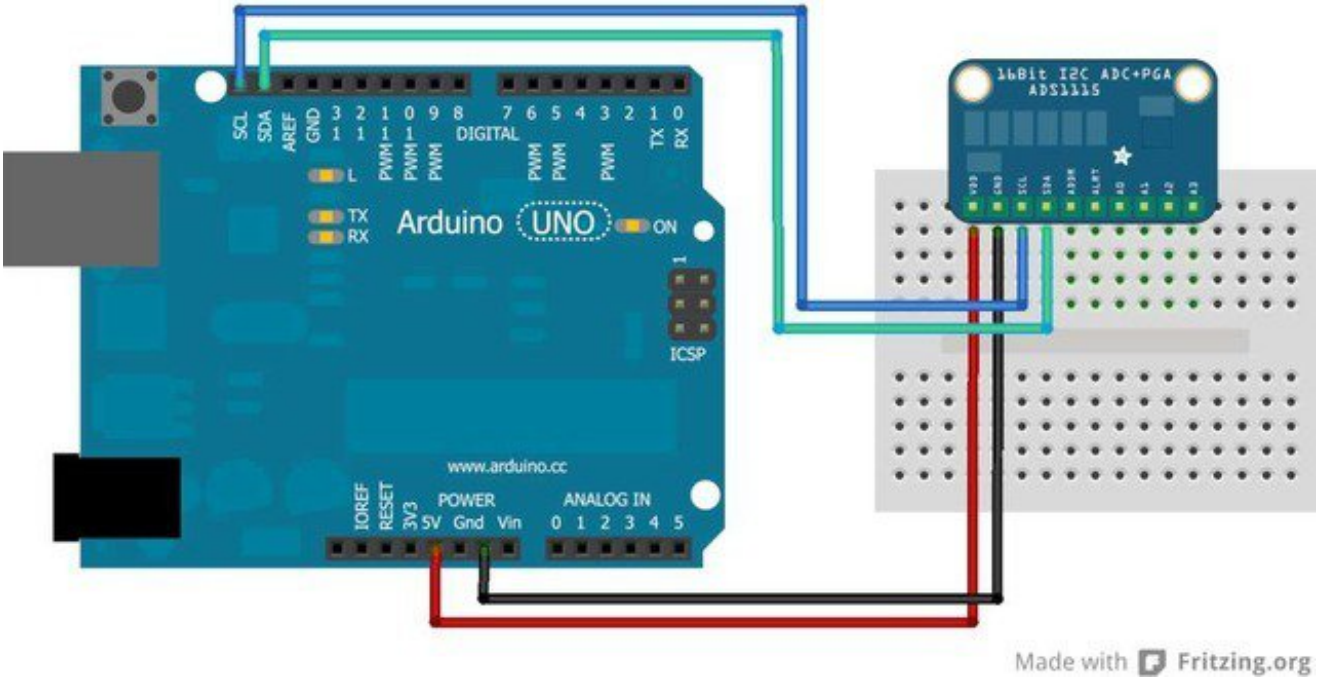
First connect VDD and GND. These boards will work with either a 3.3v or a 5v supply. The diagram below shows connection to the Arduino 5v pin.
The absolute maximum analog input voltage is VDD + 0.3v. To avoid damage to the chip, do not attempt to measure voltages greater than VDD.



I2C Connections

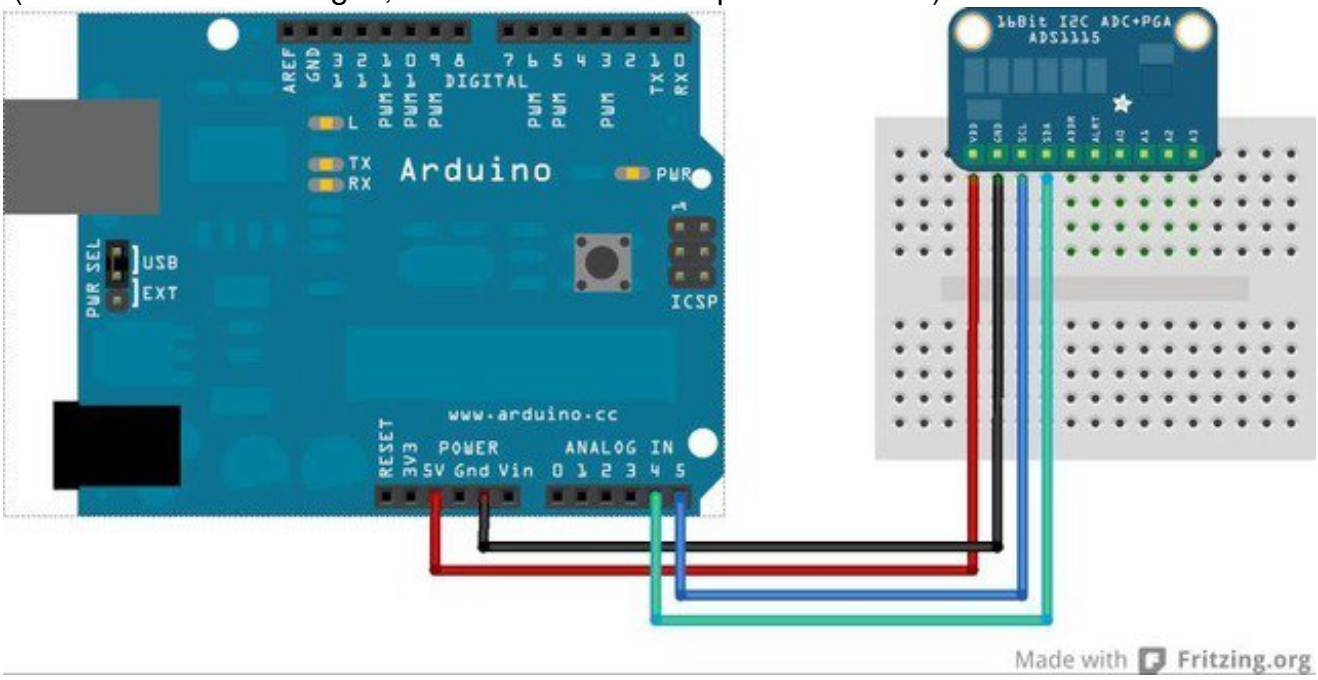
I2C requires just 2 pins to communicate. These can be shared with other I2C devices. For

R3 and later Arduinos (including MEGA and DUE models), connect SDA->SDA and SCL->SCL.



I2C "Classic"

For older Arduino boards without dedicated SDA and SCL pins, connect as shown below. (For older Arduino Megas, SDA and SCL are on pins 20 and 21)

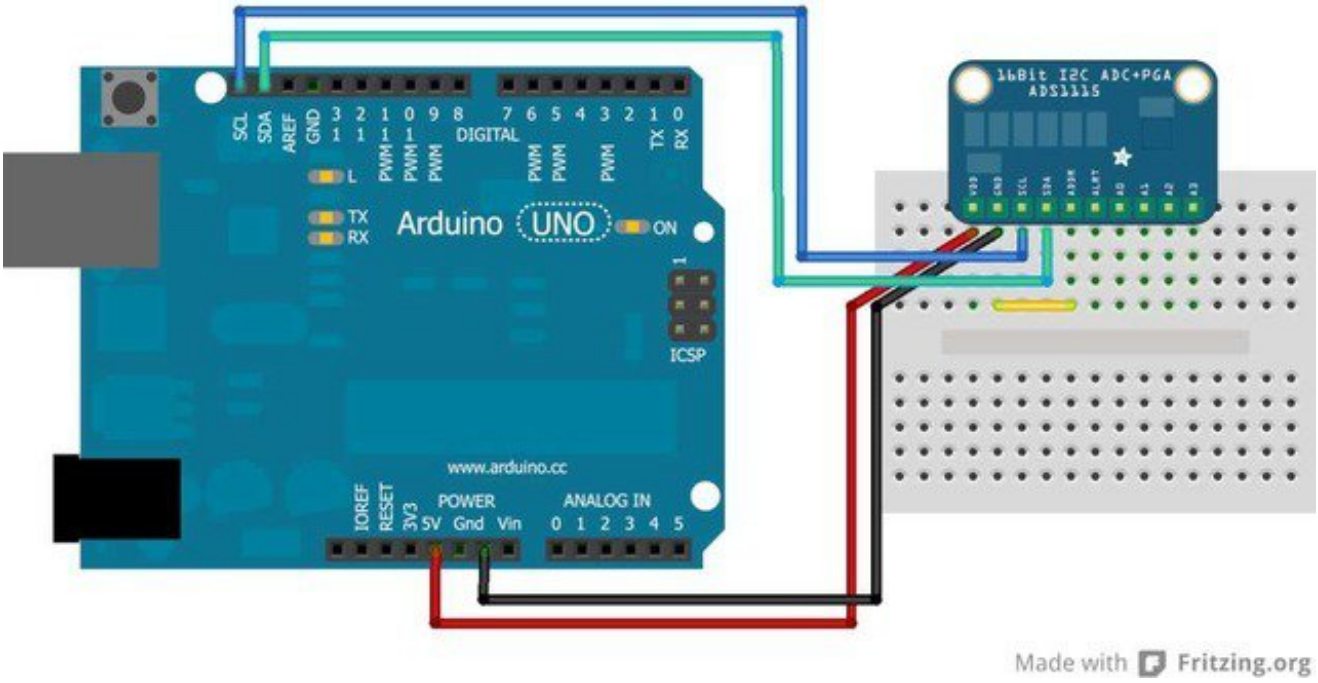


I2C Addressing

The ADS11x5 chips have a base 7-bit I2C address of 0x48 (1001000) and a clever addressing scheme that allows four different addresses using just one address pin (named **ADR** for ADdRess). To program the address, connect the address pin as follows:

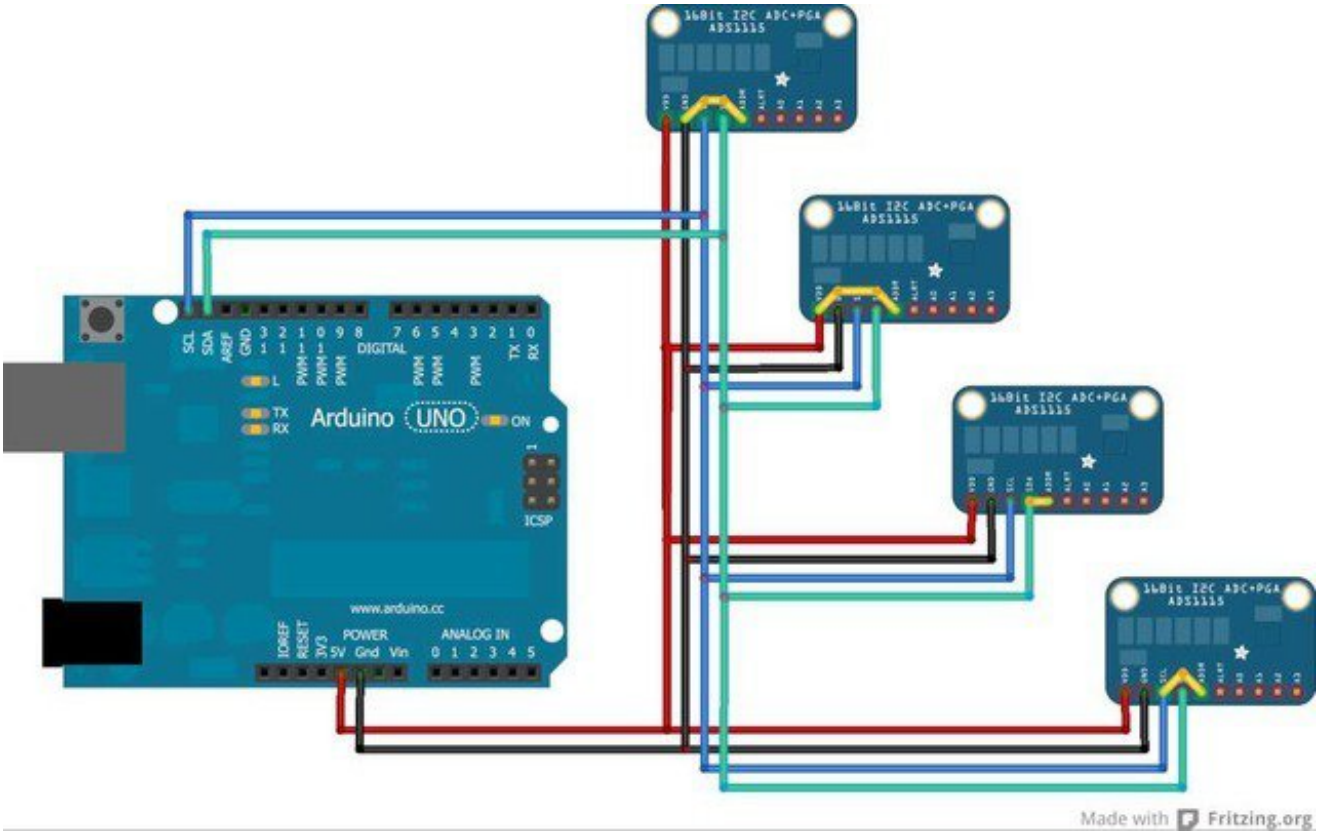
- 0x48 (1001000) ADR -> GND
- 0x49 (1001001) ADR -> VDD
- 0x4A (1001010) ADR -> SDA
- 0x4B (1001011) ADR -> SCL

The following diagram shows one board addressed as 0x48:

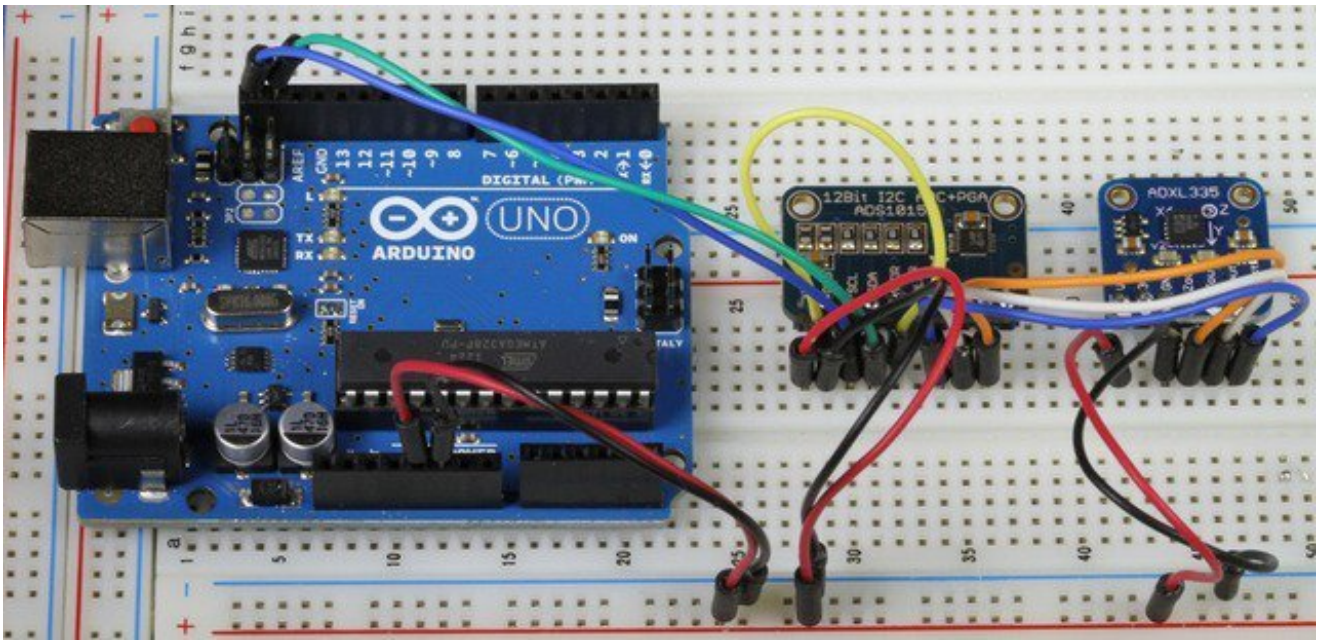


Multiple Boards

By assigning each board a different address, up to 4 boards can be connected as below:


16Bit I2C ADC+PGA
ADS1115
Arduino UNO
www.arduino.cc
Made with Fritzing.org

Signal Connections



Single Ended vs. Differential Inputs:

The ADS1x15 breakouts support up to 4 SIngle Ended or 2 Differential inputs.

Single Ended inputs measure the voltage between the analog input channel (A0-A3) and analog ground (GND).

Differential inputs measure the voltage between two analog input channels. (A0&A1 or A2&A3).

Which should I use?

Single ended inputs give you twice as many inputs. So why would you want to use differential inputs?

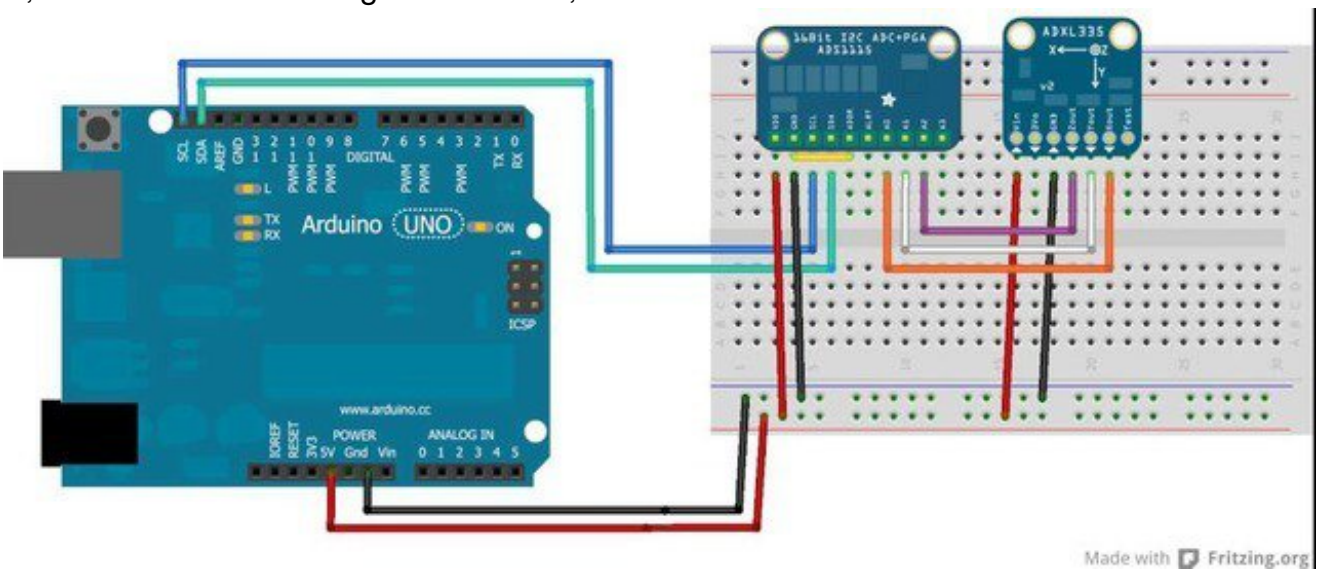
Single ended inputs can, by definition, only measure positive voltages. Without the sign bit, you only get an effective 15 bit resolution.

In addition to providing the full 16 bits of resolution and the ability to measure negative voltages, Differential measurements offer more immunity from electromagnetic noise. This

is useful when using long signal wires or operating in an electrically noisy environment. This is also desirable when dealing with small signals requiring high gain, since the gain will amplify the noise as well as the signal.

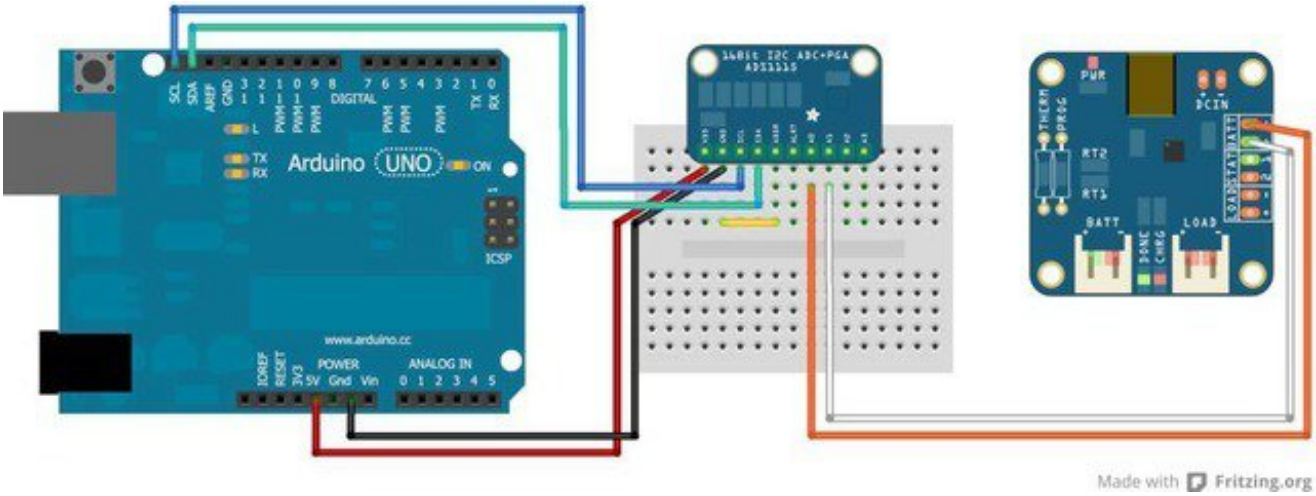
Single Ended Connections:

Connect the signal wire to one of the analog input channels (A0 - A3). Connect the ground wire to GND. This diagram shows how to connect an ADXL335 to for measurement of the X, Y and Z axis on analog channels A0, A1 and A2.



Differential Connections:

Differential measurements use a pair of input pins, either A0&A1 or A2&A3. The following diagram shows connections for differential measurement of the battery voltage on a LiPo charger board.



All input signals to these devices must be between ground potential and VCC. If your

source signal produces negative voltages, they must be offset to fall within the GND to VCC range of the ASD1x15.

Programming

The Adafruit_ADS1x15 library supports both single-ended and differential readings as well as comparator operations on both the ADS1015 and ADS1115 breakout boards. The library uses the wiring library for I2C communication, so wiring.h must be included.

Construction and Initialization:

Adafruit_ADS1015();
Construct an instance of an ADS1015 with the default address (0x48)

Adafruit_ADS1015(uint8_t addr);
Construct an instance of an ADS1015 with the specified address (0x48 - 0x4B)

Adafruit_ADS1115();
Construct an instance of an ADS1115 with the default address (0x48)

Adafruit_ADS1115(uint8_t addr);
Construct an instance of an ADS1115 with the specified address (0x48 - 0x4B)

void begin(void);
Initialize the ADC for operation.

Example:
The following examples assume an ADS1015 and use a 3 mV/bit scaling factor. For the higher-resolution ADS1115, the scaling factor would be 188uV/bit.

```
#include <Wire.h>
#include <Adafruit_ADS1015.h>

Adafruit_ADS1015 ads1015;   // Construct an ads1015 at the default address: 0x48
Adafruit_ADS1115 ads1115(0x49); // construct an ads1115 at address 0x49

void setup(void)
{
  ads1015.begin();  // Initialize ads1015
  ads1115.begin();  // Initialize ads1115
}
```

Single Ended Conversion:

uint16_t readADC_SingleEnded(uint8_t channel);

Perform a single-ended analog to digital conversion on the specified channel.

Example:

```
#include <Wire.h>
#include <Adafruit_ADS1015.h>

Adafruit_ADS1015 ads1015;

void setup(void)
{
  Serial.begin(9600);
  Serial.println("Hello!");

  Serial.println("Getting single-ended readings from AIN0..3");
  Serial.println("ADC Range: +/- 6.144V (1 bit = 3mV)");
  ads1015.begin();
}

void loop(void)
{
  int16_t adc0, adc1, adc2, adc3;

  adc0 = ads1015.readADC_SingleEnded(0);
  adc1 = ads1015.readADC_SingleEnded(1);
  adc2 = ads1015.readADC_SingleEnded(2);
  adc3 = ads1015.readADC_SingleEnded(3);
  Serial.print("AIN0: "); Serial.println(adc0);
  Serial.print("AIN1: "); Serial.println(adc1);
  Serial.print("AIN2: "); Serial.println(adc2);
  Serial.print("AIN3: "); Serial.println(adc3);
  Serial.println(" ");

  delay(1000);
}
```

Differential Conversion:

int16_t readADC_Differential_0_1(void);

Perform a differential analog to digital conversion on the voltage between channels 0 and 1.

int16_t readADC_Differential_2_3(void);

Perform a differential analog to digital conversion on the voltage between channels 2 and 3.

Example:

```
#include <Wire.h>
```

```
#include <Adafruit_ADS1015.h>

Adafruit_ADS1015 ads1015;

void setup(void)
{
  Serial.begin(9600);
  Serial.println("Hello!");

  Serial.println("Getting differential reading from AIN0 (P) and AIN1 (N)");
  Serial.println("ADC Range: +/- 6.144V (1 bit = 3mV)");
  ads1015.begin();
}

void loop(void)
{
  int16_t results;

  results = ads1015.readADC_Differential_0_1();
  Serial.print("Differential: "); Serial.print(results); Serial.print("("); Serial.print(results * 3); Serial.println("mV)");

  delay(1000);
}
```

Comparator Operation:

Comparator mode allows you to compare an input voltage with a threshold level and generate an alert signal (on the ALRT pin) if the threshold is exceeded. This pin can be polled with a digital input pin, or it can be configured to generate an interrupt.

void startComparator_SingleEnded(uint8_t channel, int16_t threshold);
Set the threshold and channel for comparator operation.

int16_t getLastConversionResults();
Get the last conversion result and clear the comparator.

Example:

```
#include <Wire.h>
#include <Adafruit_ADS1015.h>

Adafruit_ADS1015 ads1015;

void setup(void)
{
  Serial.begin(9600);
  Serial.println("Hello!");
```

```
  Serial.println("Single-ended readings from AIN0 with >3.0V comparator");
  Serial.println("ADC Range: +/- 6.144V (1 bit = 3mV)");
  Serial.println("Comparator Threshold: 1000 (3.000V)");
  ads1015.begin();

  // Setup 3V comparator on channel 0
  ads1015.startComparator_SingleEnded(0, 1000);
}

void loop(void)
{
  int16_t adc0;

  // Comparator will only de-assert after a read
  adc0 = ads1015.getLastConversionResults();
  Serial.print("AIN0: "); Serial.println(adc0);

  delay(100);
}
```

Adjusting Gain

To boost small signals, the gain can be adjusted on the ADS1x15 chips in the following steps:

- **GAIN_TWOTHIRDS** (for an input range of +/- 6.144V)
- **GAIN_ONE** (for an input range of +/-4.096V)
- **GAIN_TWO** (for an input range of +/-2.048V)
- **GAIN_FOUR** (for an input range of +/-1.024V)
- **GAIN_EIGHT** (for an input range of +/-0.512V)
- **GAIN_SIXTEEN** (for an input range of +/-0.256V)

adsGain_t getGain(void)

Reads the current gain value (default = 2/3x)

```
adsGain_t gain = getGain();
```

void setGain(adsGain_t gain)

Sets the gain for the ADS1x15

```
ads1015.setGain(GAIN_TWOTHIRDS);  // 2/3x gain +/- 6.144V  1 bit = 3mV (default)

// ads1015.setGain(GAIN_ONE);     // 1x gain   +/- 4.096V  1 bit = 2mV
// ads1015.setGain(GAIN_TWO);     // 2x gain   +/- 2.048V  1 bit = 1mV
// ads1015.setGain(GAIN_FOUR);    // 4x gain   +/- 1.024V  1 bit = 0.5mV
// ads1015.setGain(GAIN_EIGHT);   // 8x gain   +/- 0.512V  1 bit = 0.25mV
```

```
// ads1015.setGain(GAIN_SIXTEEN); // 16x gain  +/- 0.256V  1 bit = 0.125mV
```

Example

If we had an analog sensor with an output voltage ~1V (a TMP36, for example), we could set the gain on the ADC to **GAIN_FOUR**, which would give us a +/-1.024V range. This would push the 1V input signal over the entire 12-bit or 16-bit range of the ADC, compared to the very limited range 1V would cover without adjusting the gain settings

```
// Set the gain to 4x, for an input range of +/- 1.024V
// 1-bit = 0.5V on the ADS1015 with this gain setting
ads1015.setGain(GAIN_FOUR);
```

Downloads

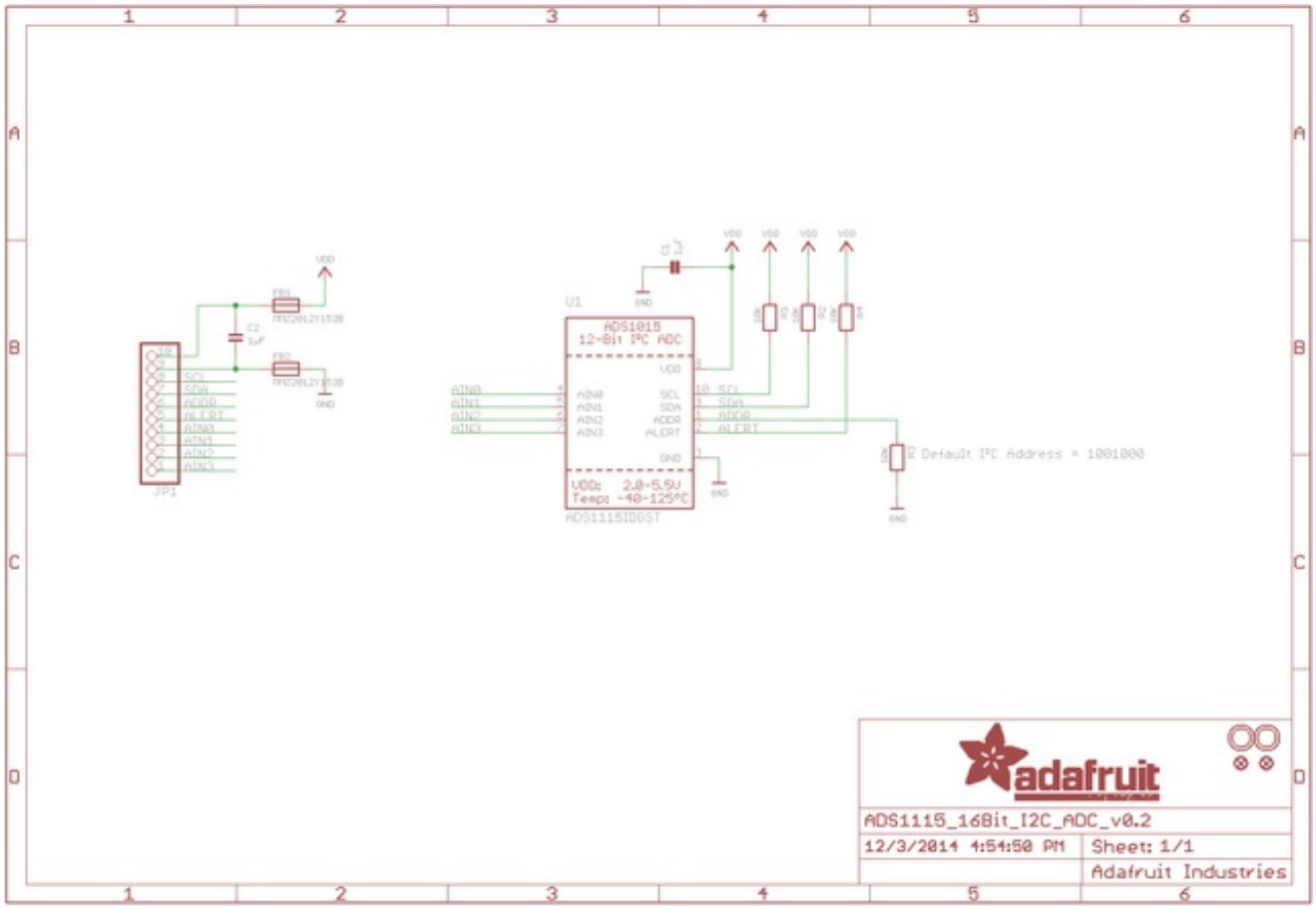
Software

- ADS1x15 Library for Arduino (http://adafru.it/aSt)

Files

- Board Files and Schematics (http://adafru.it/aSu)
- Fritzing library (http://adafru.it/aP3)
- ADS1015 Data Sheet (http://adafru.it/aSv)
- ADS1115 Data Sheet (http://adafru.it/aSw)

Schematic (Identical For Both)



Fabrication Print (Identical For Both)

